

ERP Quiz Module — Complete Study Guide

Teacher creates quizzes • Students attempt them • Real attempt data • Timing • Evaluation • Leaderboards • Analytics

Goal: Build a production-ready quiz module inside an existing ERP using a frontend, FastAPI backend, and PostgreSQL/Supabase database. This guide explains what to build, how the pieces connect, what data to store, and the order in which to implement it.

1. System Overview

Core workflow: Teacher creates quiz → adds questions → publishes quiz → student sees available quiz → starts an attempt → answers questions → answers are stored → backend evaluates the attempt → result is generated → leaderboard and analytics are updated.

```
ERP SYSTEM
■■■ Teacher Panel
■ ■■■ Create Quiz
■ ■■■ Add Questions
■ ■■■ Question Bank
■ ■■■ Publish / Schedule
■ ■■■ Analytics
■■■ Student Panel
■ ■■■ Available Quizzes
■ ■■■ Start Quiz
■ ■■■ Answer Questions
■ ■■■ Submit Quiz
■ ■■■ Results / Performance
■■■ Backend
■■■ Authentication
■■■ Quiz APIs
■■■ Attempt APIs
■■■ Evaluation
■■■ Leaderboard
■■■ Analytics
```

2. Teacher Module

Teacher Dashboard

Teacher should view created quizzes with title, subject, status, dates, attempts, and basic performance.

```
Quiz Management
[ + Create Quiz ]

My Quizzes
Quiz Subject Status
Physics Unit 1 Physics Published
Chemistry Test Chemistry Draft
Math Quiz Maths Completed
```

Create Quiz

```
Fields:
• Quiz title
• Description
• Subject / chapter
• Duration
• Total marks
• Marks per question
• Negative marking
• Start date/time
• End date/time
• Maximum attempts
• Status: Draft / Published / Closed
```

Question Creation

Support both manual entry and JSON import. JSON import is especially useful for your existing MHT-CET question extraction workflow.

```
Manual:
Question
A. Option
B. Option
C. Option
D. Option
Correct Answer
Marks
Negative Marks

Import:
questions.json → Validate → Preview → Import into Quiz
```

3. Student Quiz Experience

Show one question at a time or a clean question list with navigation. Include a visible countdown, question number, answer state, and submit action.

```
Physics Quiz
Question 7 / 20

A physical quantity has dimensions...

■ A
■ B
■ C
■ D

Time Remaining: 14:32

[Previous] [Next]

Question Navigator:
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20

Status:
Answered / Not Answered / Marked for Review
```

4. Database Design

Use PostgreSQL/Supabase as the source of truth. A relational design is essential because quiz attempts, individual answers, timing, and analytics all depend on reliable relationships.

Table	Important columns	Purpose
quizzes	id, title, teacher_id, subject_id, duration_minutes, total_marks, status	Quiz configuration, status
quiz_questions	id, quiz_id, question_text, question_type, marks, negative_marks, questions_order	Questions
question_options	id, question_id, option_key, option_text, is_correct	Answer choices
quiz_attempts	id, quiz_id, student_id, started_at, submitted_at, time_taken_seconds, student_attempt, status	One student attempt
student_answers	id, attempt_id, question_id, selected_option, is_correct, marks_obtained, answered_at, time_taken_seconds	Individual answers

5. Relationships

```
Teacher 1 ■■■■< Quizzes
Quiz 1 ■■■■< Quiz Questions
Question 1 ■■■■< Question Options
```

Student 1 ■■■■< Quiz Attempts
Quiz 1 ■■■■< Quiz Attempts
Attempt 1 ■■■■< Student Answers
Question 1 ■■■■< Student Answers

6. Real Data Storage

Save answer events as the student progresses. This improves resilience and gives accurate timing data.

```
Student selects answer
↓
POST /api/attempts/{attempt_id}/answers
↓
Validate attempt + question
↓
Store selected option + timestamp
↓
Return success
↓
Student continues
```

Important: The database should remain the source of truth. Client-side score, correctness, and final time should not be trusted.

7. Question Timing & Speed

Capture when a question becomes active and when the student answers it. Store the timing information.

```
question_opened_at
answer_selected_at
time_taken_seconds
```

Example:

```
Q1 → 4.2 sec
Q2 → 7.8 sec
Q3 → 3.1 sec
Q4 → 12.4 sec
```

```
Average Question Time =
Total valid question time / Questions attempted
```

8. Evaluation System

```
Student submits answer
↓
Backend gets correct answer from database
↓
Compare selected answer with correct answer
↓
Calculate marks
↓
Apply negative marking if configured
↓
Count correct / wrong / unanswered
↓
Calculate score + accuracy
↓
Save final attempt result
```

Never send correct answers to the student browser as part of the quiz payload. Return only question text and options. Evaluate on the server.

9. Leaderboards

Use separate leaderboard views rather than forcing every metric into one combined ranking.

Leaderboard	Metric
Score	Highest final score
Accuracy	Highest percentage of correct answers
Speed	Lowest valid average time per question
Fastest Correct	Fastest correct answer per question

For speed records, consider counting only correct answers and excluding invalid events. Otherwise students could race to click random answers.

10. Teacher Analytics

Quiz Analytics

Students Attempted: 52

Average Score: 67%

Average Accuracy: 71%

Average Time: 13m 42s

Highest Score: 96%

Question Analysis

Q1 Correct: 94% Avg Time: 18 sec

Q2 Correct: 82% Avg Time: 25 sec

Q3 Correct: 41% Avg Time: 63 sec

Q4 Correct: 76% Avg Time: 31 sec

Question-level analytics help teachers identify questions that are difficult, slow, or frequently answered incorrectly.

11. Student Analytics

My Quiz Performance

Quizzes Attempted: 18

Average Score: 78%

Average Accuracy: 81%

Average Speed: 31 sec/question

Best Score: 94%

Performance Trend

Quiz 1 → 62%

Quiz 2 → 71%

Quiz 3 → 76%

Quiz 4 → 82%

Quiz 5 → 88%

12. FastAPI Backend Structure

```
backend/
├── main.py
├── models/
│   ├── quiz.py
│   ├── question.py
│   ├── attempt.py
│   ├── answer.py
│   └── schemas/
│       ├── quiz.py
│       ├── question.py
│       └── attempt.py
├── routers/
│   ├── teacher_quiz.py
│   ├── student_quiz.py
│   ├── attempts.py
│   ├── leaderboard.py
│   ├── analytics.py
│   └── services/
│       ├── quiz_service.py
│       ├── evaluation_service.py
│       └── leaderboard_service.py
```

13. API Design

Area	Method	Endpoint	Purpose
Teacher	POST	/api/quizzes	Create quiz
Teacher	GET	/api/teacher/quizzes	List teacher quizzes
Teacher	POST	/api/quizzes/{id}/questions	Add question
Teacher	POST	/api/quizzes/{id}/publish	Publish quiz
Student	GET	/api/student/quizzes	Available quizzes
Student	POST	/api/quizzes/{id}/start	Start attempt
Attempt	POST	/api/attempts/{id}/answers	Save answer
Attempt	POST	/api/attempts/{id}/submit	Submit attempt
Results	GET	/api/attempts/{id}/result	Get result
Leaderboard	GET	/api/quizzes/{id}/leaderboard	Score leaderboard
Leaderboard	GET	/api/quizzes/{id}/speed-leaderboard	Speed leaderboard
Analytics	GET	/api/quizzes/{id}/analytics	Quiz analytics

14. Real-Time Leaderboard

For live rankings or progress, use WebSockets or Supabase Realtime. A practical design is to keep live ranking optional for students and provide a teacher-only live monitoring screen.

```
Student answers
↓
Database update
↓
Realtime event
↓
Teacher live dashboard
↓
Updated progress / rankings
```

15. Security Rules

- Authenticate every student and teacher.

- Verify that the student is allowed to attempt the quiz.
- Verify teacher permissions for quiz-management actions.
- Never send correct answers in the quiz payload.
- Calculate score on the backend.
- Calculate or validate timing on the backend.
- Enforce quiz start/end time on the server.
- Prevent unauthorized access to another student's attempt.
- Prevent duplicate attempts when only one attempt is allowed.
- Validate every question and selected option against the quiz.
- Use database constraints and transactions where appropriate.
- Do not expose sensitive student information in public leaderboards.

16. Suggested Frontend Pages

Teacher:

- /teacher/quizzes
- /teacher/quizzes/create
- /teacher/quizzes/{id}/edit
- /teacher/quizzes/{id}/questions
- /teacher/quizzes/{id}/analytics
- /teacher/question-bank

Student:

- /student/quizzes
- /student/quizzes/{id}
- /student/quiz/{attempt_id}
- /student/quiz/{attempt_id}/result
- /student/performance
- /student/leaderboard

17. Recommended UI Features

- Clean quiz cards with subject, duration, question count, and status.
- Question navigator with answered/unanswered/marked-for-review states.
- Countdown timer with auto-submit when time expires.
- Confirmation dialog before final submission.
- Result summary with score, accuracy, time, and rank.
- Separate score, accuracy, and speed leaderboard tabs.
- Teacher analytics with question-level accuracy and average time.
- Student performance trend charts.
- Responsive design for desktop and mobile.
- Clear loading, success, error, and retry states.

18. Advanced Features — Add Later

- Random question selection
- Random option order
- Question bank
- JSON import
- CSV/Excel import
- MHT-CET question bank

- Multiple attempts
- Negative marking
- Scheduled quizzes
- Auto-submit
- Mark for review
- Section-wise quizzes
- Difficulty levels
- Question tags
- Certificates
- Badges / achievements
- Subject/chapter filters
- Detailed answer review
- Teacher live monitoring
- Export results to CSV/PDF
- Reattempt rules
- Quiz access by class/division/batch

19. Implementation Roadmap

Phase	Build	Goal
1	Core Quiz	Teacher creates quiz, student attempts, result works
2	Database	Reliable storage for quizzes/questions/attempts/answers
3	Timing	Quiz timer + question timing + timestamps
4	Leaderboard	Score, accuracy, speed, fastest-correct metrics
5	Analytics	Teacher and student performance dashboards
6	Advanced	Realtime, imports, question bank, scheduling, achievements

20. Recommended Development Order

- STEP 1
Create database tables and relationships.
- STEP 2
Create teacher quiz CRUD APIs.
- STEP 3
Create student quiz listing and quiz-start API.
- STEP 4
Create attempt + answer-saving APIs.
- STEP 5
Build quiz UI with timer and navigation.
- STEP 6
Build server-side evaluation and result page.
- STEP 7
Add question timing.
- STEP 8
Build score / accuracy / speed leaderboards.
- STEP 9
Build teacher and student analytics.
- STEP 10
Add security hardening and edge-case testing.
- STEP 11
Add advanced features only after the core flow is stable.

21. Testing Checklist

- Teacher can create, edit, publish, close, and manage quizzes.
- Student can see only quizzes available to them.
- Student cannot start before the quiz start time.
- Student cannot continue after the server-side deadline.
- Timer behaves correctly after refresh/reconnect.
- Answers are saved correctly.
- Changing an answer updates the stored answer correctly.
- Score is calculated only by the backend.
- Negative marking works correctly.
- Unanswered questions are handled correctly.
- Duplicate submission does not create duplicate final results.
- Leaderboard follows your attempt-status rules.
- Teacher analytics match raw database data.
- Students cannot access other students' private results.
- Correct answers are never exposed through the quiz API.

22. Final Architecture

```

TEACHER
↓
Create Quiz → Questions → Publish
↓
POSTGRESQL / SUPABASE
↓
STUDENT
↓
Start Attempt
↓
Answer Q1 → DB
Answer Q2 → DB
Answer Q3 → DB
↓
Submit
↓
FASTAPI
■■■ Validate
■■■ Evaluate
■■■ Calculate score
■■■ Calculate accuracy
■■■ Calculate timing
■■■ Save result
↓
RESULT
■■■ Score
■■■ Accuracy
■■■ Correct / Wrong / Unanswered
■■■ Time
■■■ Rank
↓
LEADERBOARD + ANALYTICS

```

23. Key Principle to Remember

Build the data flow first, then build the visual features. If quizzes, questions, attempts, answers, timestamps, and evaluation are stored correctly, the leaderboard, charts, rankings, and analytics can all be generated from the same reliable data.

Suggested stack for your current ERP: Frontend → FastAPI → PostgreSQL/Supabase → optional Supabase Realtime/WebSockets for live updates.